# Appendices

## Table of Contents

Appendix A - Codes ..... 2
Main code ..... 2
Telegram class function ..... 6
Servo class function ..... 7
LCD class function ..... 8
Appendix B - Servo Motor ..... 11
Appendix C - Raspberry Pi Camera Module v2 ..... 13
Appendix D - ADS7830 ..... 15
Appendix E - LCD ..... 16
Appendix F - HW-104 ..... 17
Appendix G - Relay ..... 18
Appendix H - System Algorithms ..... 20
Appendix I - Explanation of Creating a Telegram Bot ..... 21

# Appendix A - Codes

## Main code

```
import cv2
from picamera2 import Picamera2
import smbus
import RPi.GPIO as GPIO
import time
import numpy as np
import sys
from threading import Thread
import threading

sys.path.append('/home/Python')
from custom_servo import Servo
from Telegram_Bot import Telegram
import LCD1602  # Import LCD1602 module for LCD integration
import os

# Initialize Picamera2 instance for camera handling
picam2 = Picamera2()

# Initialize Servo motors for pan and tilt functionality
pan = Servo(pin=13)
tilt = Servo(pin=12)

# Initial angles for pan and tilt servos
pan_angle = 0
tilt_angle = 0

# Set initial angles
pan.set_angle(pan_angle)
tilt.set_angle(tilt_angle)

# Initialize LCD1602 display with I2C address 0x27 and enable backlight
LCD1602.init(0x27, 1)

# Set GPIO mode to BCM
GPIO.setmode(GPIO.BCM)

# Define GPIO pins for relays and set them as outputs
RELAY_PIN = 27
RELAY_PIN2 = 26
GPIO.setup(RELAY_PIN, GPIO.OUT)
GPIO.setup(RELAY_PIN2, GPIO.OUT)

# Initialize I2C bus for ADC
bus = smbus.SMBus(1)  # Bus 1 indicates /dev/i2c-1
ADC_ADDRESS = 0x4b  # I2C address of the ADS7830 ADC

# Display resolution and preview configuration for camera
dispW = 400
dispH = 400
picam2.preview_configuration.main.size = (dispW, dispH)
picam2.preview_configuration.main.format = "RGB888"
picam2.preview_configuration.controls.FrameRate = 60
picam2.preview_configuration.align()
picam2.configure("preview")

# Start the camera
picam2.start()

# Initialize FPS and text properties for display
fps = 0
pos = (10, 30)
font = cv2.FONT_HERSHEY_SIMPLEX
height = 0.5
weight = 1
myColor = (0, 0, 255)
```

```
# HSV filter initial values for object tracking
hueLow = 25
hueHigh = 65
satLow = 154
satHigh = 255
valLow = 88
valHigh = 241
track = 0
myCount = 0
lcd = 3
last_speech_time = 0
last_lcd_update = 0


# Trackbar callback functions for adjusting HSV values
def onTrack1(val):
    global hueLow
    hueLow = val


def onTrack2(val):
    global hueHigh
    hueHigh = val


def onTrack3(val):
    global satLow
    satLow = val


def onTrack4(val):
    global satHigh
    satHigh = val


def onTrack5(val):
    global valLow
    valLow = val


def onTrack6(val):
    global valHigh
    valHigh = val


def onTrack7(val):
    global track
    track = val


# Create a window for the trackbars and display adjustments
cv2.namedWindow('Camera and Mask')

# Create trackbars inside the 'Camera and Mask' window
cv2.createTrackbar('Hue Low', 'Camera and Mask', hueLow, 360, onTrack1)
cv2.createTrackbar('Hue High', 'Camera and Mask', hueHigh, 360, onTrack2)
cv2.createTrackbar('Sat Low', 'Camera and Mask', satLow, 255, onTrack3)
cv2.createTrackbar('Sat High', 'Camera and Mask', satHigh, 255, onTrack4)
cv2.createTrackbar('Val Low', 'Camera and Mask', valLow, 255, onTrack5)
cv2.createTrackbar('Val High', 'Camera and Mask', valHigh, 255, onTrack6)
cv2.createTrackbar('Train-0 Track-1', 'Camera and Mask', track, 1, onTrack7)


# Function to make the Pi speak phrases in a separate thread
def say_target_in_thread(status):
    def say_target():
        if status == 1:
            os.system("espeak 'Tracking target'")
        if status == 2:
            os.system("espeak 'target destroyed'")
        if status == 3:
            os.system("espeak 'searching target'")
```

```
    # Start the speech thread
    thread = threading.Thread(target=say_target)
    thread.start()


# Function to update the LCD with a message in a separate thread
def update_lcd_in_thread(message):
    def update_lcd():
        LCD1602.clear()  # Clear previous message
        LCD1602.write(0, 0, message)

    # Start the thread
    thread = threading.Thread(target=update_lcd)
    thread.start()


# Function to send Telegram message in a separate thread
class TelegramMessageThread(Thread):
    def _init_(self):
        Thread._init_(self)

    def run(self):
        Telegram().send_message()

# Function to process frames for object detection

def process_frame(frame):
    global pan_angle, tilt_angle, track, myCount, contours, lcd, last_speech_time, last_lcd_update
    frameHSV = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    lowerBound = np.array([hueLow, satLow, valLow])
    upperBound = np.array([hueHigh, satHigh, valHigh])
    myMask = cv2.inRange(frameHSV, lowerBound, upperBound)

    # Remove noise using morphological operations
    kernel = np.ones((3, 3), np.uint8)
    myMask = cv2.dilate(myMask, kernel, iterations=1)
    myMask = cv2.erode(myMask, kernel, iterations=1)

    # Find contours in the mask
    contours, _ = cv2.findContours(myMask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    if contours:
        largest_contour = max(contours, key=cv2.contourArea)
        x, y, w, h = cv2.boundingRect(largest_contour)
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 0, 255), 2)

        if track == 1:
            GPIO.output(RELAY_PIN, GPIO.LOW)
            if myCount == 0:
                TelegramMessageThread().start()
                myCount = 1

            # Calculate pan and tilt errors for servo control
            tilt_error = (y + h / 2) - dispH / 2
            pan_error = (x + w / 2) - dispW / 2

            # Adjust tilt and pan angles with limits
            if abs(tilt_error) > 10:
                tilt_angle = max(min(tilt_angle + tilt_error / 25, 4), -90)
                tilt.set_angle(tilt_angle)

            if abs(pan_error) > 10:
                pan_angle = max(min(pan_angle - pan_error / 25, 90), -90)
                pan.set_angle(pan_angle)

            # Update LCD and speech based on tracking conditions
            if abs(pan_error) < 10 and abs(tilt_error) < 10 and lcd != 1:
                current_time = time.time()
                if current_time - last_lcd_update > 2:
                    update_lcd_in_thread("Target destroyed")
                    say_target_in_thread(2)
                    last_lcd_update = current_time
                    last_speech_time = current_time
```

```
                    lcd = 1

            if abs(pan_error) > 10 and abs(tilt_error) > 10 and lcd != 0:
                current_time = time.time()
                if current_time - last_lcd_update > 2:
                    update_lcd_in_thread("Tracking target")
                    say_target_in_thread(1)
                    last_lcd_update = current_time
                    last_speech_time = current_time
                    lcd = 0

        if track == 0:
            LCD1602.clear()
            GPIO.output(RELAY_PIN, GPIO.HIGH)

    else:
        if track == 1:
            GPIO.output(RELAY_PIN, GPIO.HIGH)
            if lcd == 3 or lcd == 1 or lcd == 0:
                current_time = time.time()
                if current_time - last_lcd_update > 5:
                    update_lcd_in_thread("Searching target")
                    say_target_in_thread(3)
                    last_lcd_update = current_time
                    last_speech_time = current_time

        myCount = 0

    return frame, myMask


# Threading class to capture frames continuously
class FrameCaptureThread(Thread):
    def _init_(self):
        Thread._init_(self)
        self.frame = None
        self.running = True

    def run(self):
        global picam2
        while self.running:
            self.frame = picam2.capture_array()

    def stop(self):
        self.running = False


# Function to read ADC value from a specified channel
def read_adc(channel):
    if channel < 0 or channel > 7:
        return -1
    command = 0x8b | (channel << 4)
    bus.write_byte(ADC_ADDRESS, command)
    adc_value = bus.read_byte(ADC_ADDRESS)
    return adc_value


# Main loop for capturing and processing frames
def main_loop():
    global fps
    frame_capture = FrameCaptureThread()
    frame_capture.start()

    last_saved_time = time.time()
    frame_counter = 1
    GPIO.output(RELAY_PIN, GPIO.HIGH)

    while True:
        tStart = time.time()
        frame = frame_capture.frame
        light_level = read_adc(0)

        if light_level < 110:
```

```
            GPIO.output(RELAY_PIN2, GPIO.HIGH)
        else:
            GPIO.output(RELAY_PIN2, GPIO.LOW)

        if frame is None:
            continue

        frame = cv2.flip(frame, -1)
        frame, myMask = process_frame(frame)

        # Resize and combine frame and mask for display
        myMask = cv2.cvtColor(myMask, cv2.COLOR_GRAY2BGR)
        combined_frame = cv2.hconcat([frame, myMask])

        # Draw a cross at the center of the frame
        centerX, centerY = dispW // 2, dispH // 2
        cv2.line(combined_frame, (centerX - 10, centerY), (centerX + 10, centerY), (0, 255, 0), 1)
        cv2.line(combined_frame, (centerX, centerY - 10), (centerX, centerY + 10), (0, 255, 0), 1)

        cv2.putText(combined_frame, str(int(fps)) + ' FPS', pos, font, height, myColor, weight)

        cv2.imshow("Camera and Mask", combined_frame)

        # Save frames every 5 seconds if tracking
        if track == 1 and contours and frame_counter < 4:
            current_time = time.time()
            if current_time - last_saved_time >= 5:
                frame_name = f"/home/aslan/Python/frame_save/frame_{frame_counter}.jpg"
                cv2.imwrite(frame_name, frame)
                print(f"Frame saved: {frame_name}")
                last_saved_time = current_time
                frame_counter += 1

        if cv2.waitKey(1) == 27 or cv2.getWindowProperty("Camera and Mask", cv2.WND_PROP_VISIBLE) < 1:
            GPIO.output(RELAY_PIN, GPIO.HIGH)
            GPIO.output(RELAY_PIN2, GPIO.LOW)
            LCD1602.clear()
            break

        tEnd = time.time()
        loopTime = tEnd - tStart
        fps = 0.9 * fps + 0.1 * (1 / loopTime)

    frame_capture.stop()
    cv2.destroyAllWindows()

# Start the main loop when the script is executed
if _name_ == "_main_":
    main_loop()
```

## Telegram class function

```
import requests


class Telegram:
    """
    A class to interact with Telegram's Bot API to send messages to a specified chat.
    """

    def _init_(self, token='***************************', chat_id='#########',
               message='target detected'):
        """
        Initializes the Telegram bot with necessary parameters.

        Parameters:
        - token (str): The bot token for authentication with the Telegram API.
        - chat_id (str): The unique identifier for the chat to send the message to.
        - message (str): The message text to send.
        """
        self.token = token
        self.chat_id = chat_id
```

```
        self.message = message
        # Construct the URL for the sendMessage API request
        self.url =
f"https://api.telegram.org/bot{self.token}/sendMessage?chat_id={self.chat_id}&text={self.message}"

    def send_message(self):
        """
        Sends the predefined message to the specified chat using the Telegram Bot API.

        Returns:
        - response.json() (dict): The response from the Telegram API in JSON format.
        """
        response = requests.get(self.url)
        return response.json()  # Returns the API response in JSON forma
```

## Servo class function

```
import pigpio

class Servo:
    """
    A class for controlling a servo motor using PWM with the pigpio library.
    """

    # Constants for pulse width limits and PWM frequency
    MAX_PW = 1250  # Maximum pulse width for +90 degrees (in microseconds)
    MIN_PW = 250   # Minimum pulse width for -90 degrees (in microseconds)
    _freq = 50     # Frequency for PWM, set to 50Hz (20ms period)

    def _init_(self, pin, min_angle=-90, max_angle=90):
        """
        Initializes the Servo instance with a specified pin and angle range.

        Parameters:
        - pin (int): GPIO pin connected to the servo.
        - min_angle (int): Minimum angle the servo can be set to (default is -90).
        - max_angle (int): Maximum angle the servo can be set to (default is +90).
        """
        # Initialize pigpio and configure the PWM pin and range
        self.pi = pigpio.pi()
        self.pin = pin
        self.pi.set_PWM_frequency(self.pin, self._freq)
        self.pi.set_PWM_range(self.pin, 10000)  # Set PWM range for finer control
        self.angle = 0
        self.max_angle = max_angle
        self.min_angle = min_angle
        # Start with no PWM signal (duty cycle of 0)
        self.pi.set_PWM_dutycycle(self.pin, 0)

    def set_angle(self, angle):
        """
        Sets the servo to the specified angle within the min and max range.

        Parameters:
        - angle (int): Desired angle to set the servo to.
        """
        # Constrain angle to the specified min and max range
        if angle > self.max_angle:
            angle = self.max_angle
        elif angle < self.min_angle:
            angle = self.min_angle
        self.angle = angle
        # Map the angle to an appropriate PWM duty cycle
        duty = self.map(angle, -90, 90, self.MIN_PW, self.MAX_PW)
        self.pi.set_PWM_dutycycle(self.pin, duty)

    def get_angle(self):
        """
        Returns the current angle of the servo.

        Returns:
        - (int): Current angle of the servo.
```

```
        """
        return self.angle

    def map(self, x, in_min, in_max, out_min, out_max):
        """
        Maps a value from one range to another.

        Parameters:
        - x (float): The input value to map.
        - in_min (float): Minimum value of the input range.
        - in_max (float): Maximum value of the input range.
        - out_min (float): Minimum value of the output range.
        - out_max (float): Maximum value of the output range.

        Returns:
        - (float): Mapped value in the output range.
        """
        return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min
```

## LCD class function

```
import time
import smbus2 as smbus

# Initialize I2C bus for communication with the LCD
BUS = smbus.SMBus(1)

def write_word(addr, data):
    """
    Writes a word (byte) of data to the specified I2C address.

    Parameters:
    - addr (int): I2C address of the device (LCD).
    - data (int): Data byte to send to the LCD.
    """
    global BLEN
    temp = data
    # Control backlight by adjusting the data byte
    if BLEN == 1:
        temp |= 0x08  # Turn backlight on
    else:
        temp &= 0xF7  # Turn backlight off
    BUS.write_byte(addr, temp)

def send_command(comm):
    """
    Sends a command to the LCD.

    Parameters:
    - comm (int): Command byte to send to the LCD.
    """
    # Send higher nibble (bit7-4) first
    buf = comm & 0xF0
    buf |= 0x04  # RS = 0, RW = 0, EN = 1
    write_word(LCD_ADDR, buf)
    time.sleep(0.002)
    buf &= 0xFB  # Set EN = 0
    write_word(LCD_ADDR, buf)

    # Send lower nibble (bit3-0) second
    buf = (comm & 0x0F) << 4
    buf |= 0x04  # RS = 0, RW = 0, EN = 1
    write_word(LCD_ADDR, buf)
    time.sleep(0.002)
    buf &= 0xFB  # Set EN = 0
    write_word(LCD_ADDR, buf)

def send_data(data):
    """
    Sends data to be displayed on the LCD.

    Parameters:
```

```
    - data (int): Data byte to send to the LCD.
    """
    # Send higher nibble (bit7-4) first
    buf = data & 0xF0
    buf |= 0x05  # RS = 1, RW = 0, EN = 1
    write_word(LCD_ADDR, buf)
    time.sleep(0.002)
    buf &= 0xFB  # Set EN = 0
    write_word(LCD_ADDR, buf)

    # Send lower nibble (bit3-0) second
    buf = (data & 0x0F) << 4
    buf |= 0x05  # RS = 1, RW = 0, EN = 1
    write_word(LCD_ADDR, buf)
    time.sleep(0.002)
    buf &= 0xFB  # Set EN = 0
    write_word(LCD_ADDR, buf)

def init(addr, bl):
    """
    Initializes the LCD with the specified I2C address and backlight setting.

    Parameters:
    - addr (int): I2C address of the LCD.
    - bl (int): Backlight enable (1 for on, 0 for off).

    Returns:
    - bool: True if initialization is successful, False otherwise.
    """
    global LCD_ADDR
    global BLEN
    LCD_ADDR = addr
    BLEN = bl
    try:
        # Initialize to 8-bit mode, then switch to 4-bit mode
        send_command(0x33)
        time.sleep(0.005)
        send_command(0x32)
        time.sleep(0.005)
        # Set display to 2-line mode and 5x7 dot format
        send_command(0x28)
        time.sleep(0.005)
        # Enable display, disable cursor
        send_command(0x0C)
        time.sleep(0.005)
        # Clear the screen
        send_command(0x01)
        # Turn on the backlight
        BUS.write_byte(LCD_ADDR, 0x08)
    except:
        return False
    else:
        return True

def clear():
    """
    Clears the LCD screen.
    """
    send_command(0x01)  # Send clear screen command

def openlight():
    """
    Turns on the LCD backlight.
    """
    BUS.write_byte(0x27, 0x08)
    BUS.close()

def write(x, y, str):
    """
    Writes a string to a specified position on the LCD.

    Parameters:
    - x (int): Column position (0-15).
```

```
    - y (int): Row position (0-1).
    - str (str): Text to display on the LCD.
    """
    # Clamp x and y to valid LCD positions
    if x < 0:
        x = 0
    if x > 15:
        x = 15
    if y < 0:
        y = 0
    if y > 1:
        y = 1

    # Calculate cursor address based on position
    addr = 0x80 + 0x40 * y + x
    send_command(addr)

    # Write each character to the LCD
    for chr in str:
        send_data(ord(chr))  # Convert character to ASCII and send

# Example usage of LCD functions
if _name_ == '_main_':
    init(0x27, 1)  # Initialize LCD at address 0x27 with backlight on
    write(4, 0, 'Hello')  # Write "Hello" at row 0, column 4
    write(7, 1, 'world!')  # Write "world!" at row 1, column 7
```

## Appendix B - Servo Motor

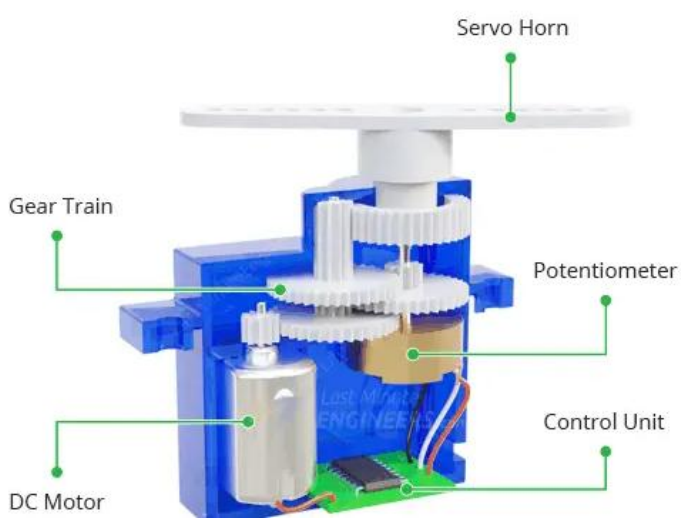
**Servo drive unit:**

A servo motor is a type of small electric motor commonly used in systems that require precise control of position, speed, or direction.

**Servo motor components:**

1. DC motor: A DC motor is used to generate rotary motion. It is connected to a gearbox that reduces the speed and increases the torque (rotational force), allowing greater precision in movement.
2. Control circuit: The control circuit is responsible for interpreting the signals sent from the external controller (such as a microcontroller or control system) and passing the instructions to the DC motor. The controller determines how far the motor should rotate or move according to the received signals.
3. Potentiometer: A potentiometer is a sensor that measures the position of the motor shaft. It is directly connected to the output shaft and therefore sends feedback to the control circuit regarding the current shaft position. Using this feedback, the controller can stop the motor when the shaft reaches the desired position.
4. Gearbox: The gearbox is connected to the motor and is used to reduce speed and generate higher torque. It allows the servo to perform fine and accurate movements.

*Servo Motor*

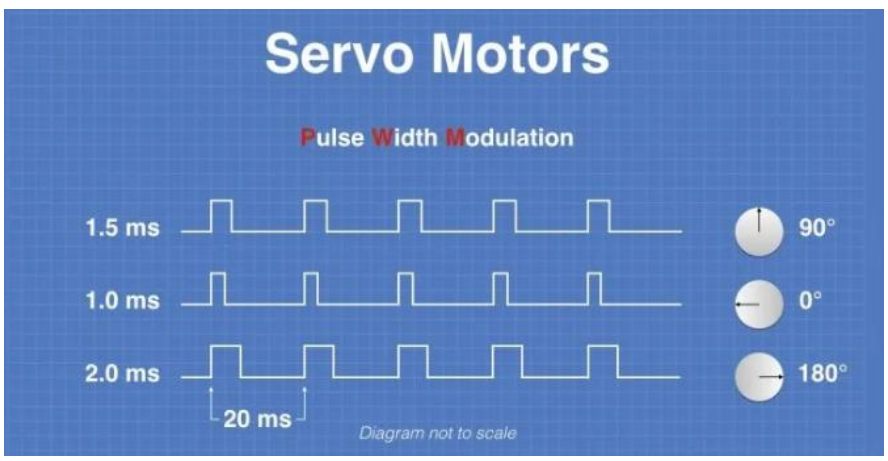


A servo motor operates based on control using Pulse Width Modulation (PWM). The servo controller receives a PWM signal that transmits pulses at a constant frequency. The pulse width defines the angle or position at which the servo motor should be set:

- A short pulse width (for example, 1 millisecond) causes the motor to move to the minimum position in its range of motion.
- A long pulse width (for example, 2 milliseconds) causes the motor to move to the maximum position.
- A pulse between 1 and 2 milliseconds places the motor at a position somewhere within the movement range according to the pulse width.

The controller sends a PWM signal approximately every 20 milliseconds. The servo control circuit measures the pulse width to determine the required position, compares it with the feedback received from the potentiometer, and then activates the motor to move the shaft to the appropriate position. When the shaft reaches the designated position, the motor stops operating.

*Pulse Width Modulation*



## Appendix C - Raspberry Pi Camera Module v2

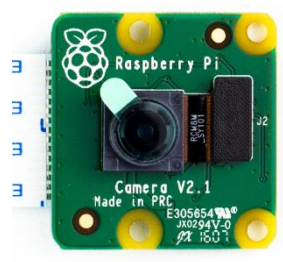
**Raspberry Pi Camera Module v2:**

The Raspberry Pi Camera Module v2 is a digital camera designed for use with Raspberry Pi computers. It is suitable for image capture, video, image processing, and object detection. The camera has a horizontal field of view (FOV) of 62.2 degrees and a vertical FOV of 48.8 degrees.

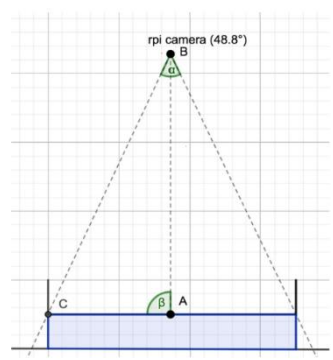
**Main features:**

1. Image sensor:
   - Equipped with a Sony IMX219 CMOS sensor.
   - 8-megapixel resolution (3280 x 2464 pixels), providing sharp and clear images.
   - Supports HD video capture.
2. System interface:
   - Connects to the Raspberry Pi through the CSI interface for fast data transfer.
   - Installation is simple: just connect the camera cable to the Raspberry Pi.
3. Support for image processing libraries:
   - Supported by libraries such as OpenCV and Picamera, enabling the development of advanced applications such as face detection, object tracking, and motion detection.

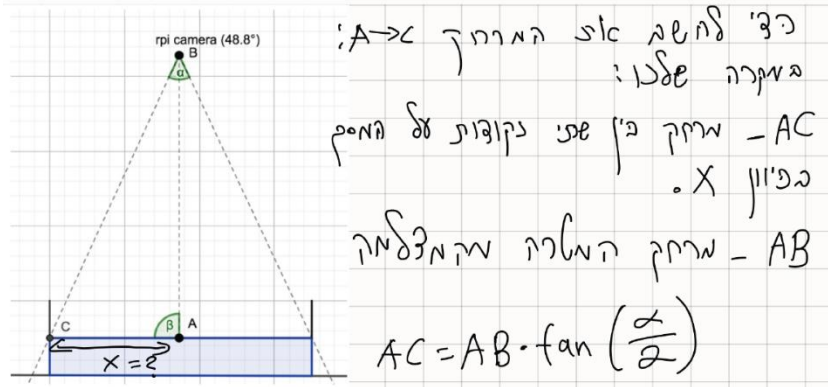
*Raspberry Pi Camera Module v2*




*FOV*



The FOV (Field of Vision) of the Raspberry Pi v2 camera is 62.2 degrees in the horizontal direction and 48.8 degrees in the vertical direction. The following calculation considers the vertical direction:




```
def pixel_to_cm(pix_x, pix_y, obj_distance, dispW, dispH, fov_x=62.2, fov_y=48.8):
    # Calculate the angular displacement per pixel
    angle_per_pixel_x = fov_x / dispW  # Degrees per pixel in X direction
    angle_per_pixel_y = fov_y / dispH  # Degrees per pixel in Y direction
    # Total angular displacement in both directions
    angle_x = pix_x * angle_per_pixel_x
    angle_y = pix_y * angle_per_pixel_y
    # Convert angular displacement to actual distance using tangent function
    dist_x_cm =  obj_distance * math.tan(math.radians(angle_x / 2))
    dist_y_cm =  obj_distance * math.tan(math.radians(angle_y / 2))
    return dist_x_cm, dist_y_cm
```

## Appendix D - ADS7830

The ADS7830 ADC (Analog-to-Digital Converter) is an 8-channel analog-to-digital converter. It is suitable for use in systems that require reading analog signals and converting them into digital values so that they can be read and processed by microcontrollers or other systems.

**Main features of the ADS7830:**

1. 8-bit resolution: The converter provides an 8-bit result for each reading, allowing precision of 256 different value levels.
2. 8 multiplexed input channels: The ADC includes eight channels, so eight sensors or different signal sources can be connected.
3. I2C communication protocol: The ADS7830 uses the I2C protocol to communicate with the microcontroller or system, simplifying the connection and enabling convenient control.
4. Low operating voltage: The voltage required to operate the component is between 2.7 and 5.5 volts, making it suitable for use with various microcontrollers and sensors.
5. Sampling rate: The converter can perform sampling at rates of up to 200 kilo-samples per second, which is suitable for applications requiring fast signal reading.

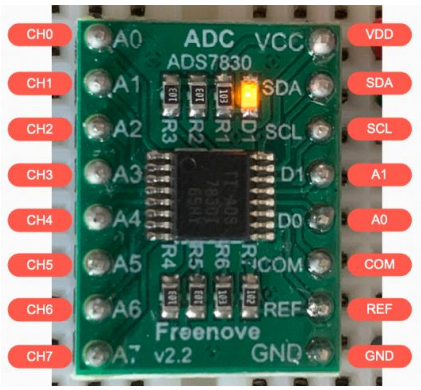
**Principle of operation**

The ADS7830 operates by receiving the analog voltage from each channel, converting it into an 8-bit digital value, and sending the information to the microcontroller through the I2C protocol. Each channel can be selected through a unique I2C command configuration, and the active channel can be determined at any given moment.

**Advantages and applications**

The ADS7830 is especially suitable for applications where it is necessary to read values from several analog sensors and transfer the data digitally to a microcontroller. For example:

- Reading temperature values from multiple temperature sensors.
- Reading signals from humidity, light, or any other analog sensor.
- Control and monitoring systems that require fast reading of analog signals.



## Appendix E - LCD

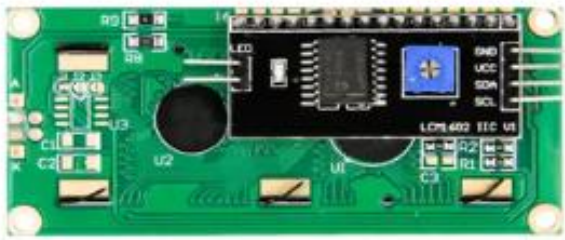
An LCD screen with an I2C interface is a display that can be connected to a microcontroller or digital system using the I2C communication protocol, which greatly simplifies the process of connecting and controlling the display. When using the I2C interface, only 4 wires are needed: two power wires (VCC and GND) and two communication wires (SDA and SCL), which allow information to be transferred to the microcontroller and enable control of the display.

**Advantages of an LCD with I2C**

1. Connection simplicity: Instead of using a large number of wires to control all the pins of the LCD, I2C uses only two wires for communication, freeing additional microcontroller pins for other uses.
2. Remote control: The display can be connected at some distance from the microcontroller and controlled through the I2C communication line.
3. Unique address: Every I2C device has a unique address, so multiple I2C devices (such as additional sensors) can be connected on the same communication line.

**Structure and principle of operation**

Standard LCD screens are usually connected to a dedicated I2C module called PCF8574 or a similar component. The module functions as an adapter that enables I2C communication with the display and simplifies the control process. It receives commands from the microcontroller and translates them into commands that the LCD can understand.

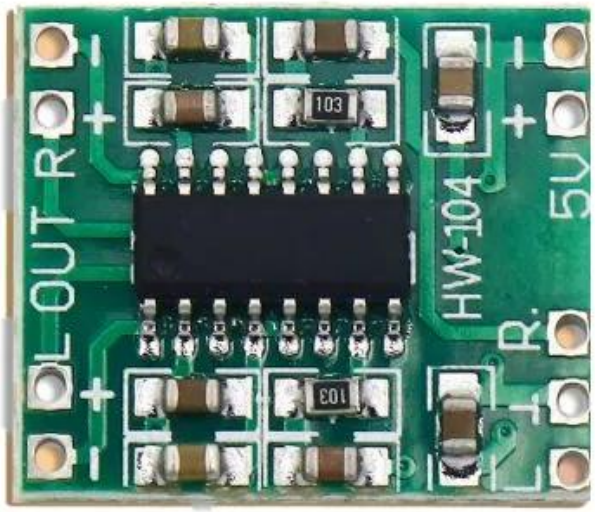


## Appendix F - HW-104

The HW-104 is a text-to-speech conversion module that enables embedded systems to generate synthesized speech from text. This module is suitable for a variety of applications, such as smart home systems, robotics, voice navigation systems, and more.

**Main features:**

- Support for multiple languages: The module supports various languages, including English, Chinese, and additional languages, depending on the version.
- Communication interface: The HW-104 communicates with microcontrollers or other systems via a UART interface, allowing easy integration into existing systems.
- Speech quality: The module produces clear and pleasant-to-hear speech, with the option to adjust speed and pitch.
- Ease of use: Integrating the module into a system requires basic power and communication connections, and the control commands are simple to implement.



## Appendix G - Relay

A relay is an electromechanical component used to connect or disconnect an electrical circuit by means of an external control signal, without directly connecting the circuits to each other. A relay operates like a switch, but instead of a handle or push-button, the control for its activation comes from an electrical signal.

**Relay structure and principle of operation**

**A relay includes several main components:**

1. Coil: When electric current flows through the coil, a magnetic field is created around it, which pulls or pushes the moving parts inside the relay.
2. Contacts: The contacts in a relay are the parts that connect or disconnect the circuit. In the normal state (when the coil is not energized), the contacts can be either open or closed, depending on the relay type.
3. Armature or moving contact: This part moves due to the magnetic field and activates the contacts in the relay.

**Types of relays**

**There are several types of relays designed for different applications:**

1. Normally Open (NO) relay: The relay is disconnected in its normal state, and when the coil is energized, the contact closes and allows current to pass.
2. Normally Closed (NC) relay: The relay is connected in its normal state, and when the coil is energized, the contact opens and current stops flowing.
3. Changeover relay: A relay containing contacts that switch between open and closed states, making it possible to connect two different circuits to different contacts.

**Common applications of relays**

**Relays are used in a variety of industrial, household, and electronic applications, especially when there is a need to switch higher loads or operate as controlled switches:**

- Controlling high loads: Relays allow small circuits and microcontrollers to control circuits that require higher voltage and current.
- Automation systems: The relay is used for automatic operation of electrical devices such as lighting, air conditioners, or heating systems.
- Electrical isolation: The relay makes it possible to separate the controlling circuit from the controlled circuit, providing electrical isolation and preventing dangerous currents from passing between circuits.

**Advantages and disadvantages**

**Advantages:**

- Enables control of high loads by means of low current.
- Provides electrical isolation between circuits.
- Simple and durable over time.

**Disadvantages:**

- Requires voltage to energize the coil.
- May generate noise due to contact movement.
- In circuits with high frequency or repeated switching, contact wear may occur.

## Appendix H - System Algorithms

Several algorithms were used in the project to achieve the system objectives:

1. Object Tracking with Image Processing

The main algorithm used is intended for color-based object tracking. The algorithm performs the following steps:

- Image input from the camera: The camera captures a sequence of frames.
- Conversion to HSV: Each frame is converted into the HSV color space, which is more convenient for color-based image processing.
- Color filtering: Lower and upper color limits (defined using the trackbars) are used in order to filter out irrelevant colors and leave only objects in the desired color.
- Mask processing: The mask generated by the filter undergoes morphological operations such as dilate and erode to clean noise and obtain a better mask.
- Contour detection: The algorithm finds the contours of the object based on the mask.
- Object position calculation: Once a contour is found, the algorithm calculates the coordinates of the object (x, y) and draws a rectangle around it.

2. Servo Motor Control for Object Tracking

Once the object is detected, the algorithm calculates the deviation of the object from the center of the image:

- Tilt Error: The difference between the center of the object and the vertical axis of the image. If there is a significant error (greater than 10 pixels), the system changes the angle of the servo motor that moves the camera vertically.

$$\text{tilt_error} = (y + h/2) - dispH/2$$

y - the y coordinate of the upper-left corner of the rectangle surrounding the object.

h - the height of the rectangle.

dispH - the height resolution of the selected camera.

- Pan Error: The difference between the center of the object and the horizontal axis of the image. If the error is greater than 10 pixels, the system changes the angle of the servo motor that moves the camera horizontally.

$$pan_\text{error} = (x + w/2) - dispW/2$$

x - the x coordinate of the upper-left corner of the rectangle surrounding the object.

w - the width of the rectangle.

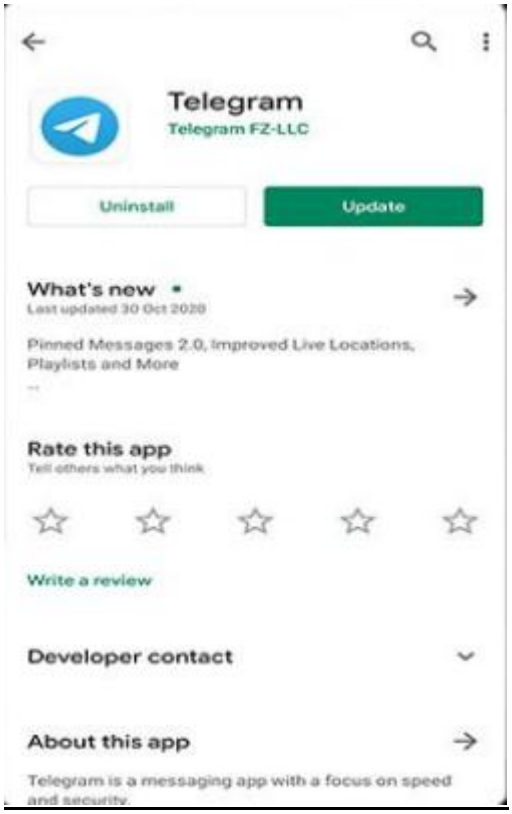
dispW - the width resolution of the selected camera.

3. Use of the Telegram Module for Sending an Alert

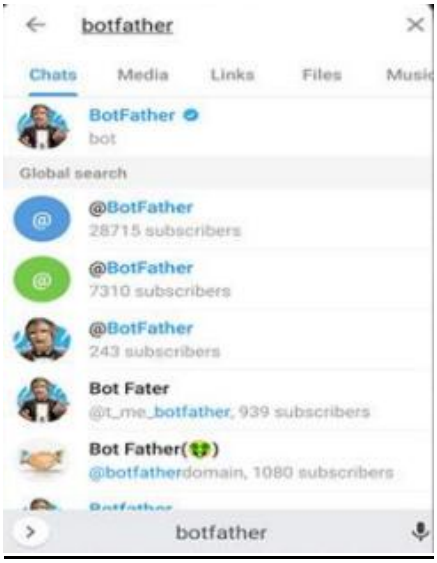
When the object is detected, the algorithm activates the TelegramMessageThread class, which sends an alert message through the Telegram API reporting target detection.

## Appendix I - Explanation of Creating a Telegram Bot

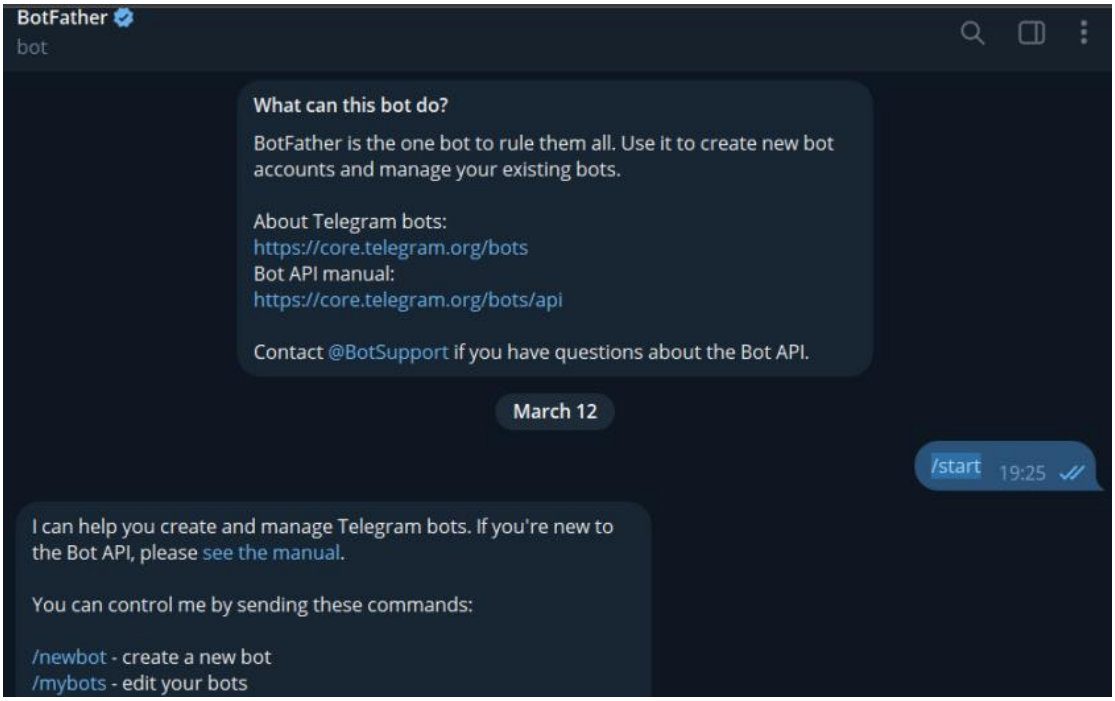
1. Installing the Telegram platform and registering an account:




2. Starting a conversation with BOTFATHER:

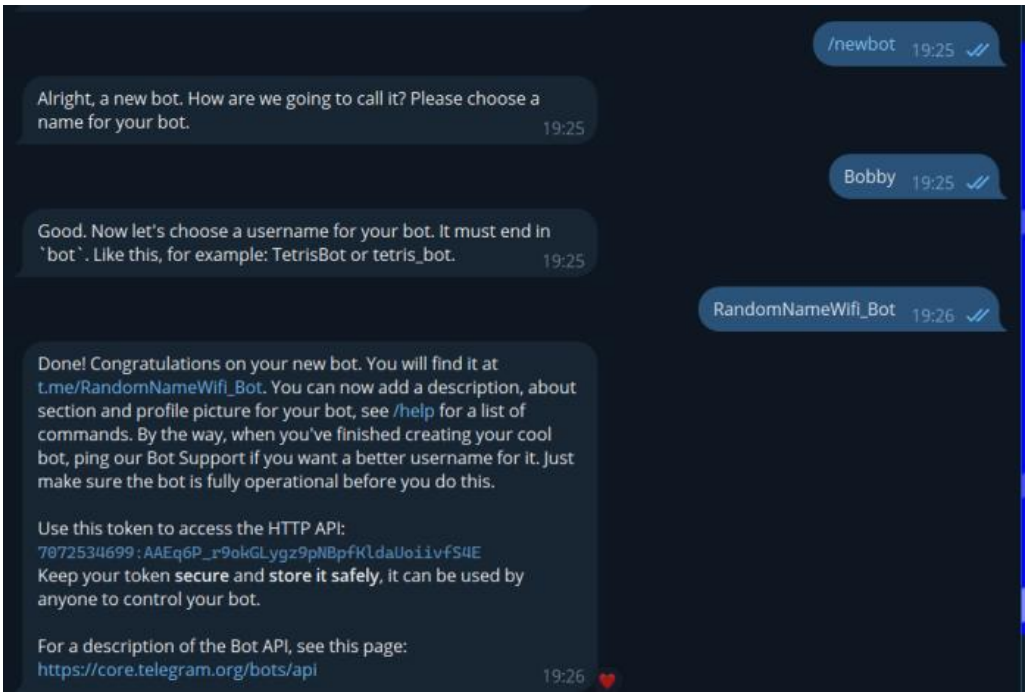


BOTFATHER is an existing bot that creates bots for your use. Send /start and then /newbot, and then define a name and a username for the bot.

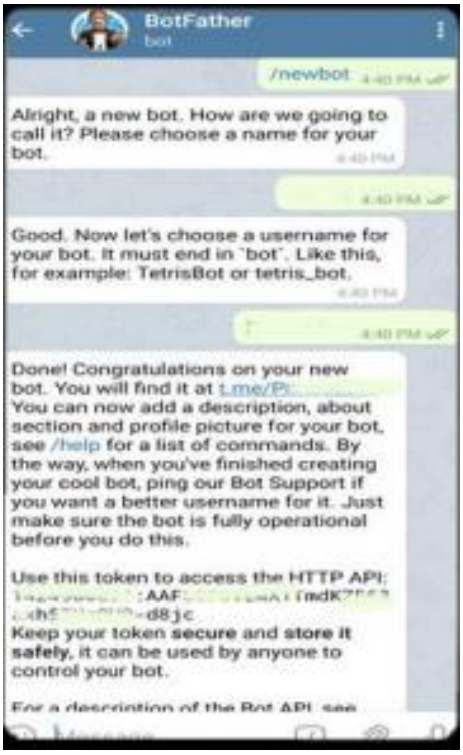



3. Press the /start button and select the /newbot option:

Choose a name and a username for the bot.



4. Once a new bot has been successfully created, a link containing the "bot ID" will be sent to us. This identifier will be used to communicate between the component and the bot.




5. Obtaining the chat ID:

The chat ID is a unique number for each chat. It is possible to access the bot through a public link, but in order to prevent unauthorized access, the communication can be secured by using the chat ID.

Type /getid in the chat to receive the ID.